

UART-Controlled Register File & ALU System

Simulation Waveform Analysis - Commands, Frames and Responses

6

simulated scenarios

4

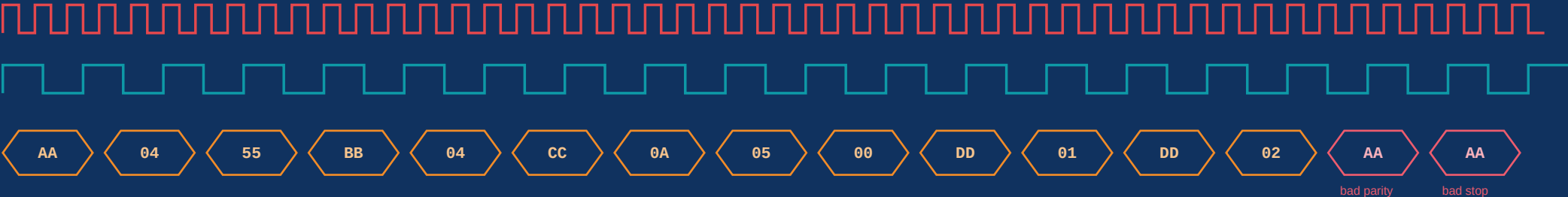
command types

2

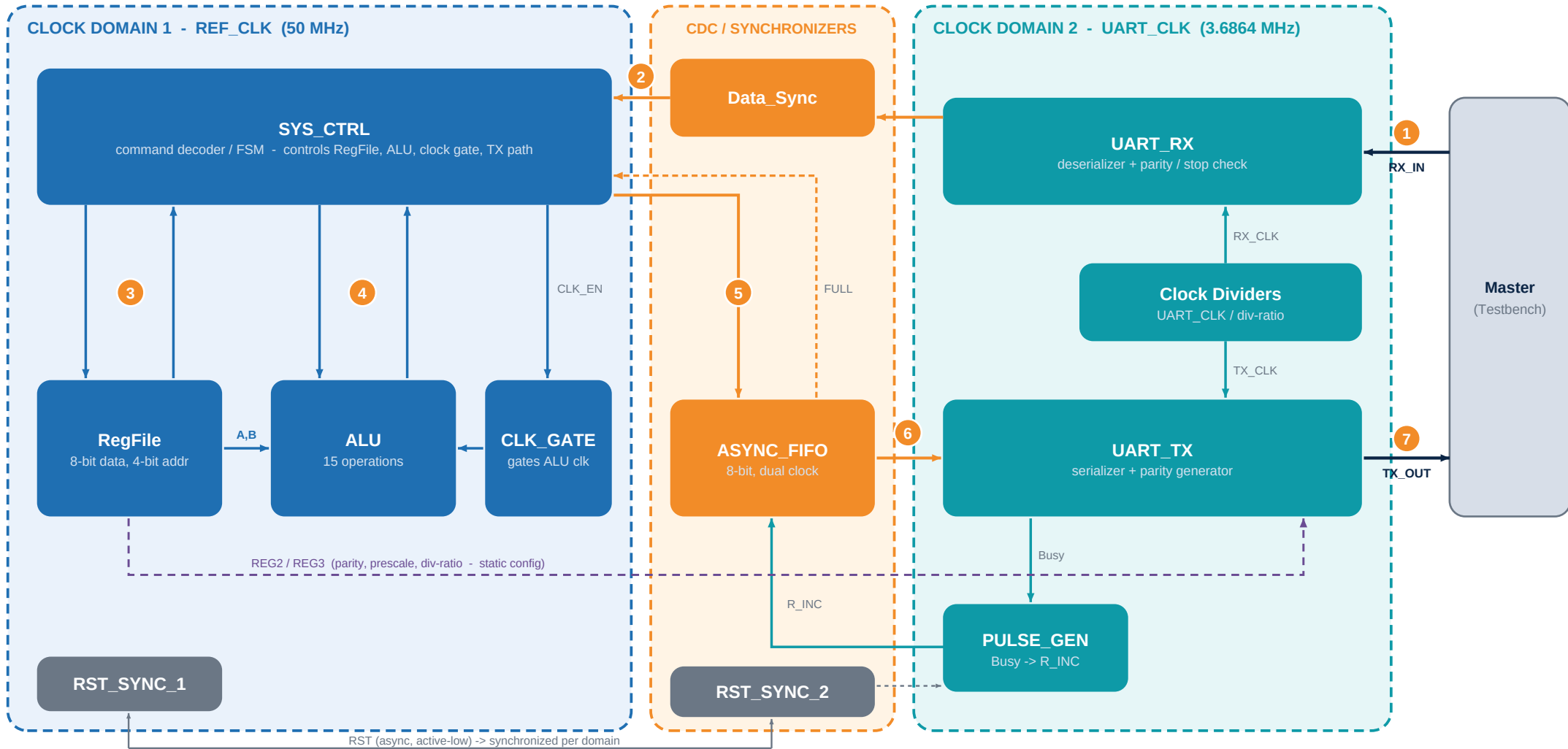
clock domains

11 bit

UART frame



Commands enter through UART_RX, are interpreted by SYS_CTRL, executed by RegFile / ALU, and the result returns through ASYNC_FIFO and UART_TX.



- 1 RX_IN: serial command frames from the master
- 2 RX_P_DATA[7:0] + RX_D_VLD (UART -> REF domain via Data_Sync)
- 3 WrEn / RdEn / Address / WrData -> RegFile; RdData + Valid back
- 4 ALU_FUN + EN -> ALU (operands = REG0, REG1); ALU_OUT[15:0] + OUT_VALID back

- 5 WR_DATA + WR_INC -> ASYNC_FIFO (REF -> UART domain); FULL back
- 6 RD_DATA + EMPTY -> UART_TX; PULSE_GEN creates R_INC from Busy
- 7 TX_OUT: serial response frames back to the master
- - - static configuration (REG2 / REG3)

System Specifications & UART Frame

Clocking, serial format and register map used by the testbench

SPECS

Key specifications

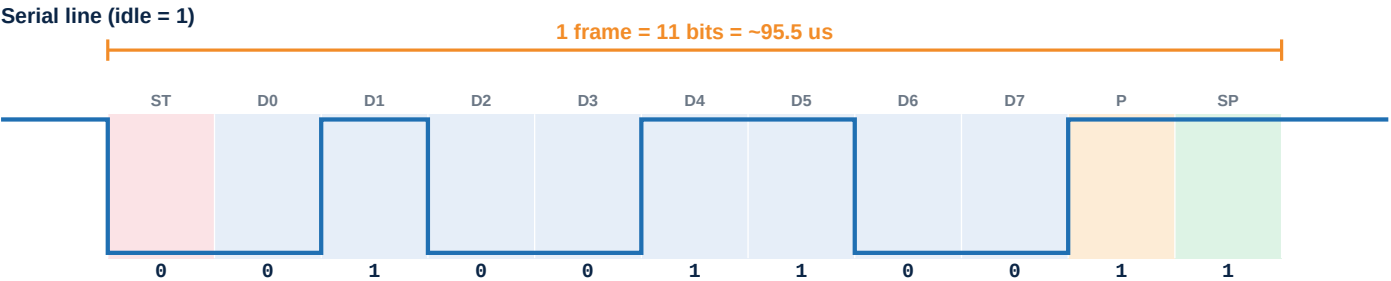
REF_CLK	50 MHz (SYS_CTRL, RegFile, ALU, FIFO write side)
UART_CLK	3.6864 MHz (UART_TX / UART_RX, clock dividers)
Clock divider	always enabled (enable = 1)
Baud rate (derived)	~115.2 kbps = 3.6864 MHz / 32
Frame format	1 start + 8 data + 1 parity + 1 stop = 11 bits
Bit time / frame time	~8.68 us / ~95.5 us (matches waveform)
Parity (defaults)	enabled, even (REG2[0]=1, REG2[1]=0)
RegFile	8-bit data, 4-bit address; 0x0-0x3 reserved
General R/W range	0x4 to 0xF (4-bit address, 16 x 8-bit registers)

Register map (reserved registers)

Reg	Addr	Description
REG0	0x0	ALU operand A
REG1	0x1	ALU operand B
REG2	0x2	UART config: [0] Parity Enable (def 1) [1] Parity Type (def 0) [7:2] Prescale (def 32)
REG3	0x3	Clock-divider ratio [7:0] (def 32)

Normal writes / reads from the testbench use addresses 0x4 and above.
REG2 / REG3 keep their reset defaults (0x81 / 0x20) - no config write is sent.
REG0 / REG1 feed the ALU directly as operands A and B.

UART frame structure (example: byte 0x32, even parity, LSB first)



ST Start bit = 0

D0-D7 8 data bits, LSB first

P Even parity of the data byte

SP Stop bit = 1

Frames of one command are sent back-to-back (the stop bit is followed directly by the next start bit); the testbench adds a ~10 us gap only between the RF write and the RF read in Test 1. The receiver samples the line at the divided UART clock; a wrong parity bit raises parity_error and a low stop bit raises stop_error, and the corrupted frame is dropped. Both flags stay low in Tests 1-4; Tests 5 and 6 raise them on purpose.

Supported Commands

COMMANDS

Frame layout of each command (master -> system) and the response (system -> master)

CMD ADDR DATA OPERAND FUN RESP

RF Write

Opcode 0xAA

3 frames

Writes DATA into RegFile[ADDR]. Silent - no response frame is sent back.



Master -> RX_IN

no response frame

RF Read

Opcode 0xBB

2 frames

Reads RegFile[ADDR]. The value comes back as one UART frame.



Master -> RX_IN



TX_OUT -> Master

ALU with operand

Opcode 0xCC

4 frames

Loads A and B into REG0 / REG1, then runs the ALU function. 16-bit result = 2 frames.



Master -> RX_IN



TX_OUT -> Master

ALU no operand

Opcode 0xDD

2 frames

Reuses the operands already stored in REG0 / REG1. 16-bit result = 2 frames.

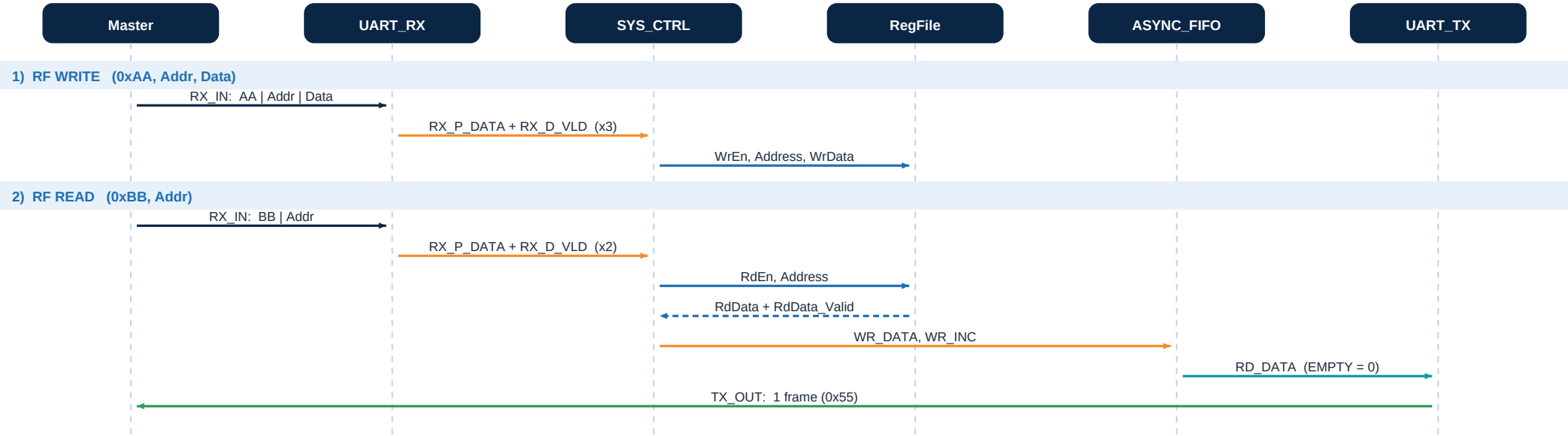


Master -> RX_IN

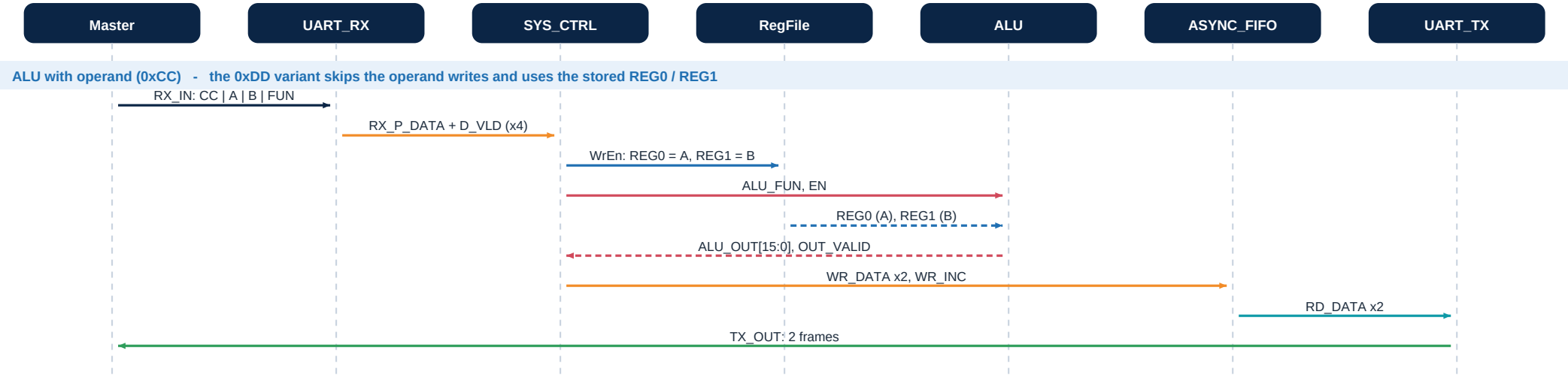


TX_OUT -> Master

Register File write and read



ALU operation



Test 1 - RF Write (0xAA) then RF Read (0xBB)

Command definition, FSM path, decoded response and event timeline

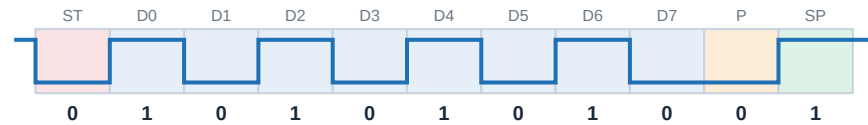
Window 0 - 600 us

1. Command frames sent by the master (RX_IN)



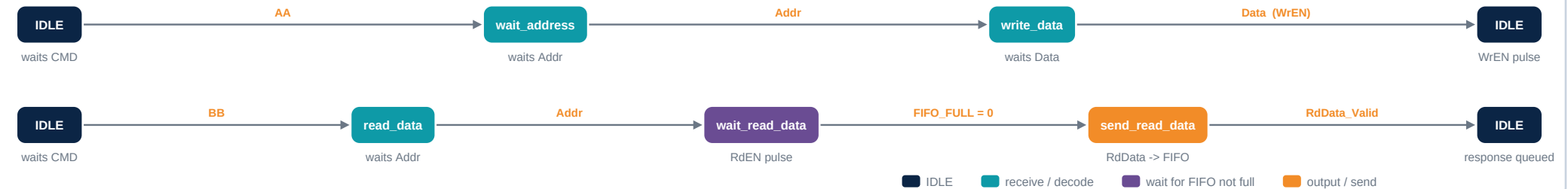
3. Response decoded from TX_OUT

Response frame 0: 0x55 (even parity bit = 0)



Bits are shown LSB first. ST = start, P = parity, SP = stop.

2. SYS_CTRL FSM path (states taken from SYS_CTRL.sv)



4. Verification checklist

- ✓ One WrEN pulse and one RdEN pulse
- ✓ Read-back value 0x55 equals the written value
- ✓ Exactly one response frame on TX_OUT
- ✓ No ALU activity (EN stays low)
- ✓ parity_error = 0 and stop_error = 0
- ✓ FSM returns to IDLE after the write and after the read

5. Event timeline (times read from the waveform, in us)

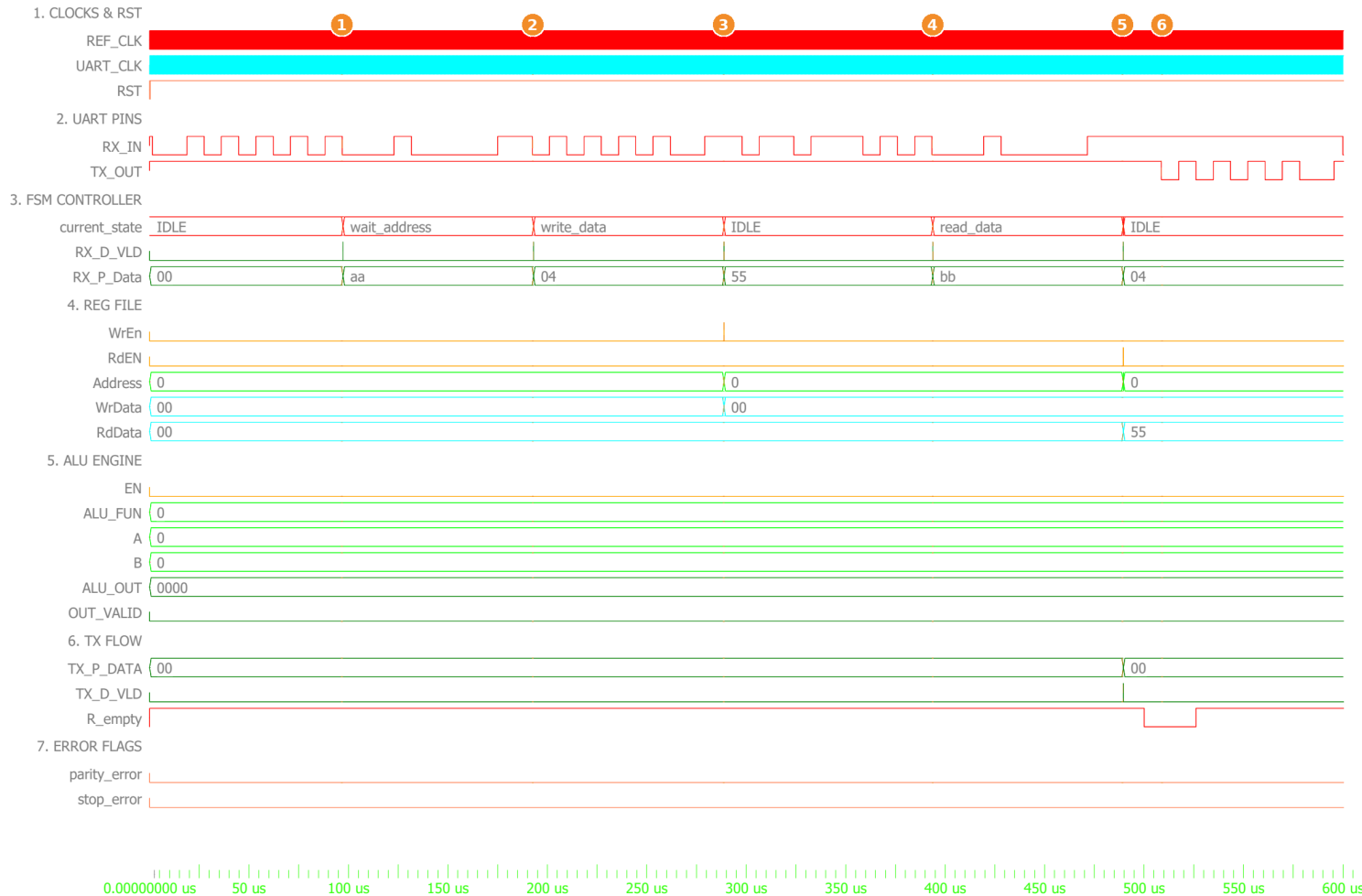
Time (us)	Event	Observed behaviour
~96	Frame 0 = 0xAA received	RX_D_VLD pulse; FSM IDLE -> wait_address
~192	Frame 1 = 0x04 (address)	FSM wait_address -> write_data
~288	Frame 2 = 0x55 (data)	WrEN pulse: RegFile[0x04] = 0x55; FSM write_data -> IDLE
~394	Frame 0 = 0xBB received	FSM IDLE -> read_data
~489	Frame 1 = 0x04 (address)	FSM read_data -> wait_read_data -> send_read_data -> IDLE; RdEN pulse, RdData = 0x55 pushed to FIFO
~500-525	FIFO not empty	R_empty goes low until UART_TX pops the byte
~509-600	Response frame	TX_OUT serial frame carrying 0x55 (alternating 1-0-1-0 pattern)

RESULT WRITE RegFile[0x04] = 0x55 then READ RegFile[0x04] -> 0x55 = MATCH

Test 1 - Waveform

Annotated simulation output (system_tb)

Window 0 - 600 us



Entity:system_tb Architecture: Date: Sat Oct 03 23:13:44 +0300 2026 Row: 1 Page: 1

Key events

- ~96 us**
0xAA captured - FSM enters wait_address
- ~192 us**
Address 0x04 captured - FSM enters write_data
- ~288 us**
Data 0x55 captured - WrEN pulse writes RegFile[0x04]; FSM back to IDLE
- ~394 us**
0xBB captured - FSM enters read_data
- ~489 us**
Address 0x04 captured - wait_read_data -> send_read_data, RdEN pulse, RdData = 0x55
- ~509 us**
UART_TX starts the response frame (0x55)

How to read this waveform

Sections 1-7 from top to bottom: clocks and reset, UART pins, SYS_CTRL FSM, register file, ALU engine, TX flow, error flags. Dashed orange lines mark the numbered events listed on the right. The RX_IN trace carries the command frames; the TX_OUT trace carries the response. RX_P_Data and the FSM state change each time a complete frame has been received (RX_D_VLD pulse). Decimal values (A = 10, B = 5) and hexadecimal values (0x0A, 0x05) refer to the same data. The intermediate states wait_read_data / send_read_data last only a few REF_CLK cycles (20 ns each), so the viewer shows them as thin slices inside the IDLE label.

Test 2 - ALU with operand: ADD (A = 10, B = 5)

Command definition, FSM path, decoded response and event timeline

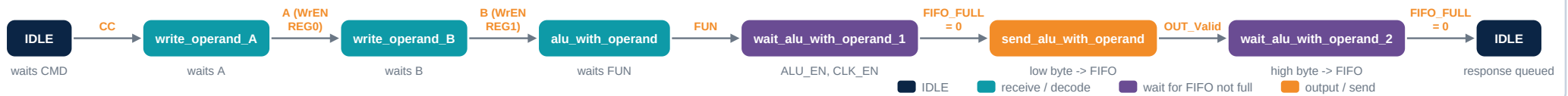
Window 650 - 1200 us

1. Command frames sent by the master (RX_IN)



0xCC = ALU with operand: A and B are written to REG0 / REG1 first, then FUN selects the operation (0x00 = ADD).

2. SYS_CTRL FSM path (states taken from SYS_CTRL.sv)



4. Verification checklist

- ✓ A = 10 and B = 5 written into REG0 / REG1
- ✓ ALU_OUT = 0x000F = 10 + 5
- ✓ OUT_VALID asserted together with the EN pulse
- ✓ Two response frames: 0x0F then 0x00
- ✓ parity_error = 0 and stop_error = 0
- ✓ FSM returns to IDLE after the high byte is queued

5. Event timeline (times read from the waveform, in us)

Time (us)	Event	Observed behaviour
~698	Frame 0 = 0xCC received	FSM IDLE -> write_operand_A
~796	Frame 1 = 0x0A (operand A)	WrEN pulse: REG0 = 10; FSM write_operand_A -> write_operand_B
~894	Frame 2 = 0x05 (operand B)	WrEN pulse: REG1 = 5; FSM write_operand_B -> alu_with_operand
~991	Frame 3 = 0x00 (ALU_FUN = ADD)	FSM alu_with_operand -> wait_alu_with_operand_1; FIFO_FULL = 0: ALU_EN + CLK_EN asserted
~991 +	ALU executes	FSM -> send_alu_with_operand: ALU_OUT = 0x000F, OUT_VALID = 1; low byte 0x0F pushed to FIFO
~991 +	High byte queued	FSM -> wait_alu_with_operand_2: high byte 0x00 pushed to FIFO; FSM -> IDLE
~1000-1134	FIFO not empty	R_empty low while the two result bytes are popped
~1009-1200	Response frames	TX_OUT: 0x0F (low byte) then 0x00 (high byte)

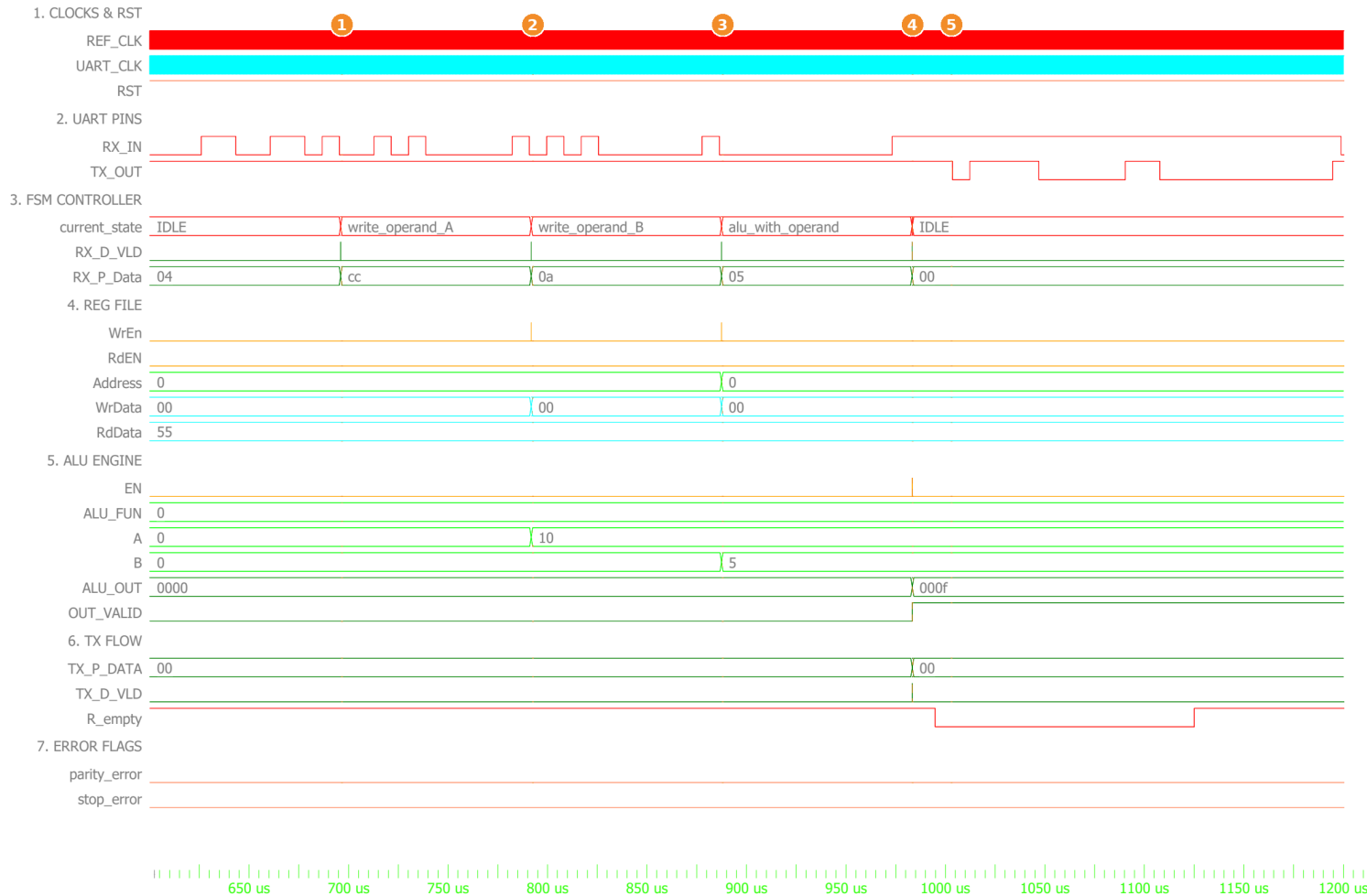
"+" = a few REF_CLK cycles (20 ns each) later; the wait_* / send_* states are too short to be labelled at this zoom.

RESULT A = 0x0A (10) + B = 0x05 (5) = ALU_OUT 0x000F (15) -> TX_OUT: 0F 00

Test 2 - Waveform

Annotated simulation output (system_tb)

Window 650 - 1200 us



Entity:system_tb Architecture: Date: Sat Oct 03 23:16:34 +0300 2026 Row: 1 Page: 1

Key events

- ~698 us**
0xCC captured - FSM enters write_operand_A
- ~796 us**
Operand A = 0x0A captured - WrEn writes REG0
- ~894 us**
Operand B = 0x05 captured - WrEn writes REG1
- ~991 us**
ALU_FUN = 0x00 captured -
wait_alu_with_operand_1 ->
send_alu_with_operand, EN pulse, ALU_OUT = 0x000F
- ~1009 us**
Response starts on TX_OUT (0x0F, 0x00)

How to read this waveform

Sections 1-7 from top to bottom: clocks and reset, UART pins, SYS_CTRL FSM, register file, ALU engine, TX flow, error flags. Dashed orange lines mark the numbered events listed on the right. The RX_IN trace carries the command frames; the TX_OUT trace carries the response. RX_P_Data and the FSM state change each time a complete frame has been received (RX_D_VLD pulse). Decimal values (A = 10, B = 5) and hexadecimal values (0x0A, 0x05) refer to the same data. The intermediate wait_* / send_* states last only a few REF_CLK cycles (20 ns each), so they are too thin to be labelled at this zoom.

Test 3 - ALU without operand: SUB (A - B = 10 - 5)

Command definition, FSM path, decoded response and event timeline

Window ~1200 - 1600 us

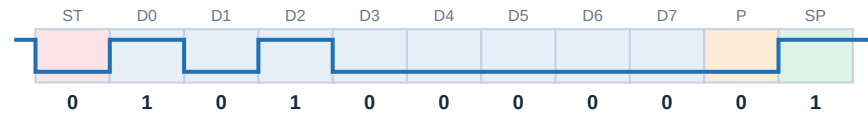
1. Command frames sent by the master (RX_IN)



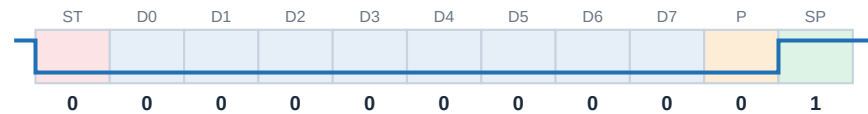
0xDD = ALU without operand: reuses REG0 / REG1 (A = 10, B = 5); FUN = 0x01 selects SUB.

3. Response decoded from TX_OUT

Response frame 0: 0x05 (even parity bit = 0)



Response frame 1: 0x00 (even parity bit = 0)



Bits are shown LSB first. ST = start, P = parity, SP = stop.

2. SYS_CTRL FSM path (states taken from SYS_CTRL.sv)



4. Verification checklist

- ✓ Operands reused from RegFile (A = 10, B = 5)
- ✓ Command completes in only 2 frames
- ✓ ALU_OUT = 0x0005 = A - B = 10 - 5
- ✓ Two response frames: 0x05 then 0x00
- ✓ parity_error = 0 and stop_error = 0
- ✓ No WrEN pulse - RegFile is not rewritten

5. Event timeline (times read from the waveform, in us)

Time (us)	Event	Observed behaviour
~1295	Frame 0 = 0xDD received	FSM IDLE -> alu_without_operand
~1391	Frame 1 = 0x01 (ALU_FUN = SUB)	FSM alu_without_operand -> wait_alu_without_operand_1; FIFO_FULL = 0: ALU_EN + CLK_EN asserted
~1391 +	ALU executes	FSM -> send_alu_without_operand: ALU_OUT = 0x0005, OUT_VALID = 1; low byte 0x05 pushed to FIFO
~1391 +	High byte queued	FSM -> wait_alu_without_operand_2: high byte 0x00 pushed to FIFO; FSM -> IDLE
~1404-1533	FIFO not empty	R_empty low while the result bytes are popped
~1412-1600	Response frames	TX_OUT: 0x05 (low byte) then 0x00 (high byte)

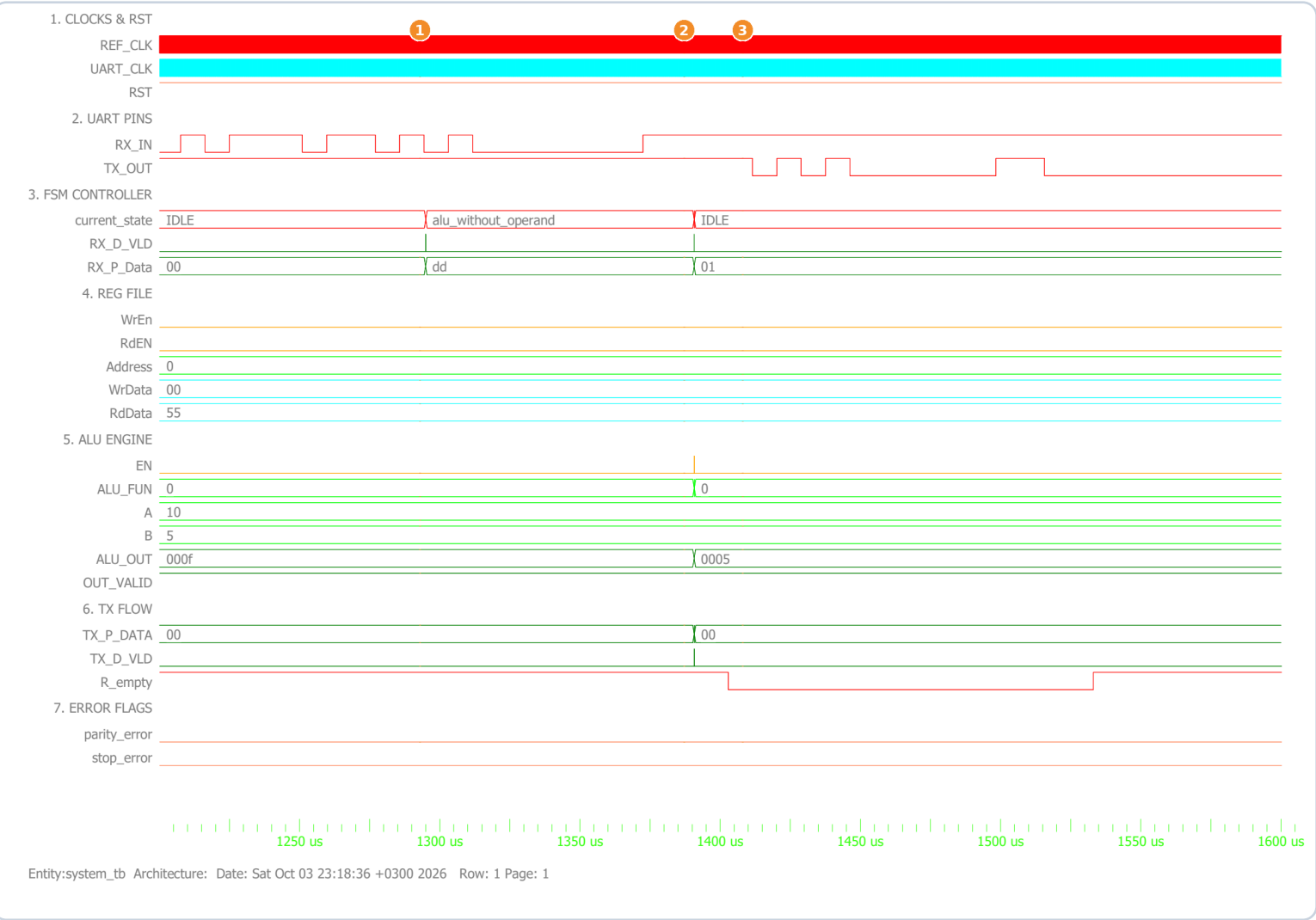
"+" = a few REF_CLK cycles (20 ns each) later; the wait_* / send_* states are too short to be labelled at this zoom.

RESULT A = 10 - B = 5 = ALU_OUT 0x0005 (5) -> TX_OUT: 05 00

Test 3 - Waveform

Annotated simulation output (system_tb)

Window ~1200 - 1600 us



Key events

1

~1295 us
0xDD captured - FSM enters alu_without_operand

2

~1391 us
ALU_FUN = 0x01 captured - wait_alu_without_operand_1 -> send_alu_without_operand, EN pulse, ALU_OUT = 0x0005

3

~1412 us
Response starts on TX_OUT (0x05, 0x00)

How to read this waveform

Sections 1-7 from top to bottom: clocks and reset, UART pins, SYS_CTRL FSM, register file, ALU engine, TX flow, error flags. Dashed orange lines mark the numbered events listed on the right. The RX_IN trace carries the command frames; the TX_OUT trace carries the response. RX_P_Data and the FSM state change each time a complete frame has been received (RX_D_VLD pulse). Decimal values (A = 10, B = 5) and hexadecimal values (0x0A, 0x05) refer to the same data. The intermediate wait_* / send_* states last only a few REF_CLK cycles (20 ns each), so they are too thin to be labelled at this zoom.

Test 4 - ALU without operand: MUL (A x B = 10 x 5)

Command definition, FSM path, decoded response and event timeline

Window ~1600 - 2000 us

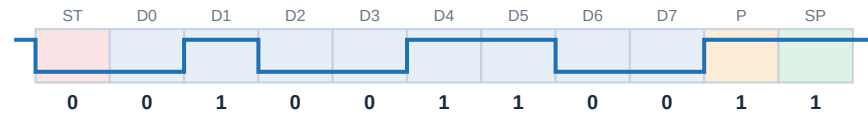
1. Command frames sent by the master (RX_IN)



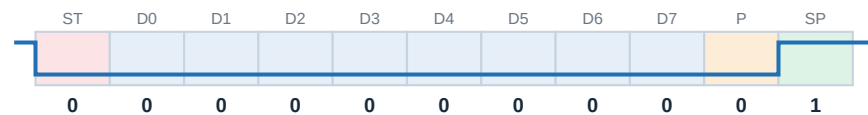
0xDD = ALU without operand: reuses REG0 / REG1 (A = 10, B = 5); FUN = 0x02 selects MUL.

3. Response decoded from TX_OUT

Response frame 0: 0x32 (even parity bit = 1)



Response frame 1: 0x00 (even parity bit = 0)



Bits are shown LSB first. ST = start, P = parity, SP = stop.

2. SYS_CTRL FSM path (states taken from SYS_CTRL.sv)



4. Verification checklist

- ✓ Operands reused from RegFile (A = 10, B = 5)
- ✓ Command completes in only 2 frames
- ✓ ALU_OUT = 0x0032 = A x B = 10 x 5
- ✓ Two response frames: 0x32 then 0x00
- ✓ parity_error = 0 and stop_error = 0
- ✓ No WrEN pulse - RegFile is not rewritten

5. Event timeline (times read from the waveform, in us)

Time (us)	Event	Observed behaviour
~1703	Frame 0 = 0xDD received	FSM IDLE -> alu_without_operand
~1799	Frame 1 = 0x02 (ALU_FUN = MUL)	FSM alu_without_operand -> wait_alu_without_operand_1; FIFO_FULL = 0: ALU_EN + CLK_EN asserted
~1799 +	ALU executes	FSM -> send_alu_without_operand: ALU_OUT = 0x0032, OUT_VALID = 1; low byte 0x32 pushed to FIFO
~1799 +	High byte queued	FSM -> wait_alu_without_operand_2: high byte 0x00 pushed to FIFO; FSM -> IDLE
~1810-1942	FIFO not empty	R_empty low while the result bytes are popped
~1820-2000	Response frames	TX_OUT: 0x32 (low byte) then 0x00 (high byte)

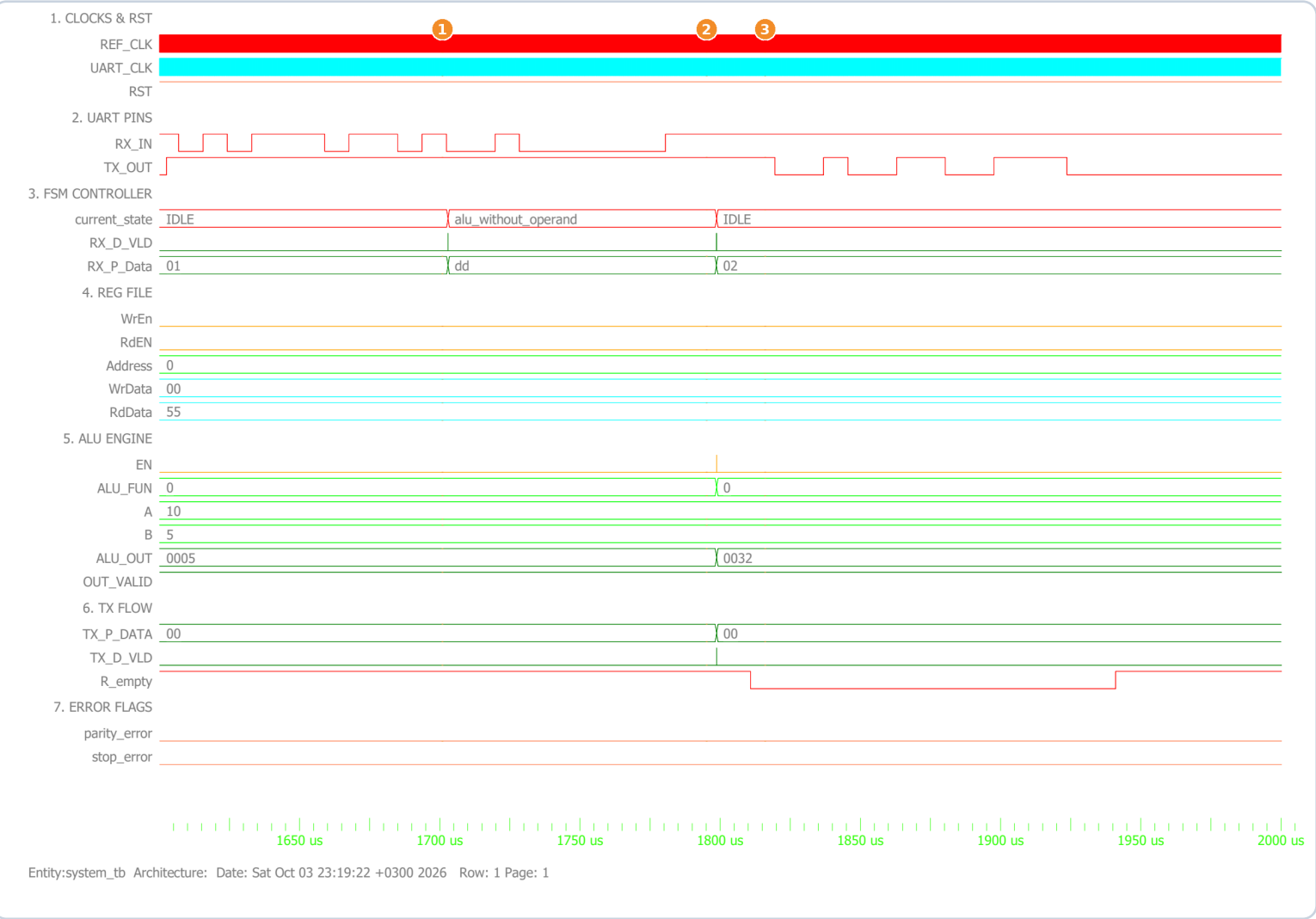
"+" = a few REF_CLK cycles (20 ns each) later; the wait_* / send_* states are too short to be labelled at this zoom.

RESULT A = 10 x B = 5 = ALU_OUT 0x0032 (50) -> TX_OUT: 32 00

Test 4 - Waveform

Annotated simulation output (system_tb)

Window ~1600 - 2000 us



Key events

- 1 ~1703 us**
0xDD captured - FSM enters alu_without_operand
- 2 ~1799 us**
ALU_FUN = 0x02 captured - wait_alu_without_operand_1 -> send_alu_without_operand, EN pulse, ALU_OUT = 0x0032
- 3 ~1820 us**
Response starts on TX_OUT (0x32, 0x00)

How to read this waveform

Sections 1-7 from top to bottom: clocks and reset, UART pins, SYS_CTRL FSM, register file, ALU engine, TX flow, error flags. Dashed orange lines mark the numbered events listed on the right. The RX_IN trace carries the command frames; the TX_OUT trace carries the response. RX_P_Data and the FSM state change each time a complete frame has been received (RX_D_VLD pulse). Decimal values (A = 10, B = 5) and hexadecimal values (0x0A, 0x05) refer to the same data. The intermediate wait_* / send_* states last only a few REF_CLK cycles (20 ns each), so they are too thin to be labelled at this zoom.

Test 5 - Error injection: parity error (0xAA, bad parity bit)

Window ~1950 - 2150 us

Command definition, UART_RX FSM path, injected frame and event timeline

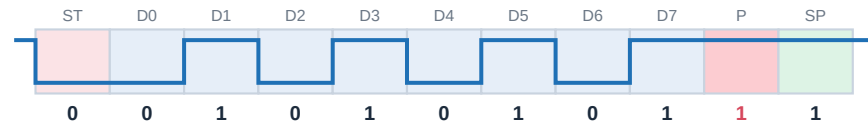
1. Command frame sent by the master (RX_IN)



0xAA = RF write command. It is never executed: the frame is rejected by the parity check. Corrupted bit shown in red on the right.

3. Frame injected on RX_IN (corrupted bit highlighted)

Injected frame 0xAA: parity bit forced to 1 (expected 0)



Bits are shown LSB first. ST = start, P = parity, SP = stop. Idle line = 1.

2. UART_RX FSM_RX path (rejected frame)



SYS_CTRL: stays in IDLE - RX_D_VLD never pulses, the corrupted frame is dropped inside UART_RX (FSM_RX).

4. Verification checklist

- ✓ parity_error = 1 after the frame (~2098 us)
- ✓ stop_error stays 0
- ✓ No RX_D_VLD pulse - frame rejected
- ✓ SYS_CTRL stays IDLE: no Wren / RdEN / ALU EN
- ✓ No response frame on TX_OUT (R_empty stays high)
- ✓ Testbench self-check prints PASS

5. Event timeline (times read from the waveform, in us)

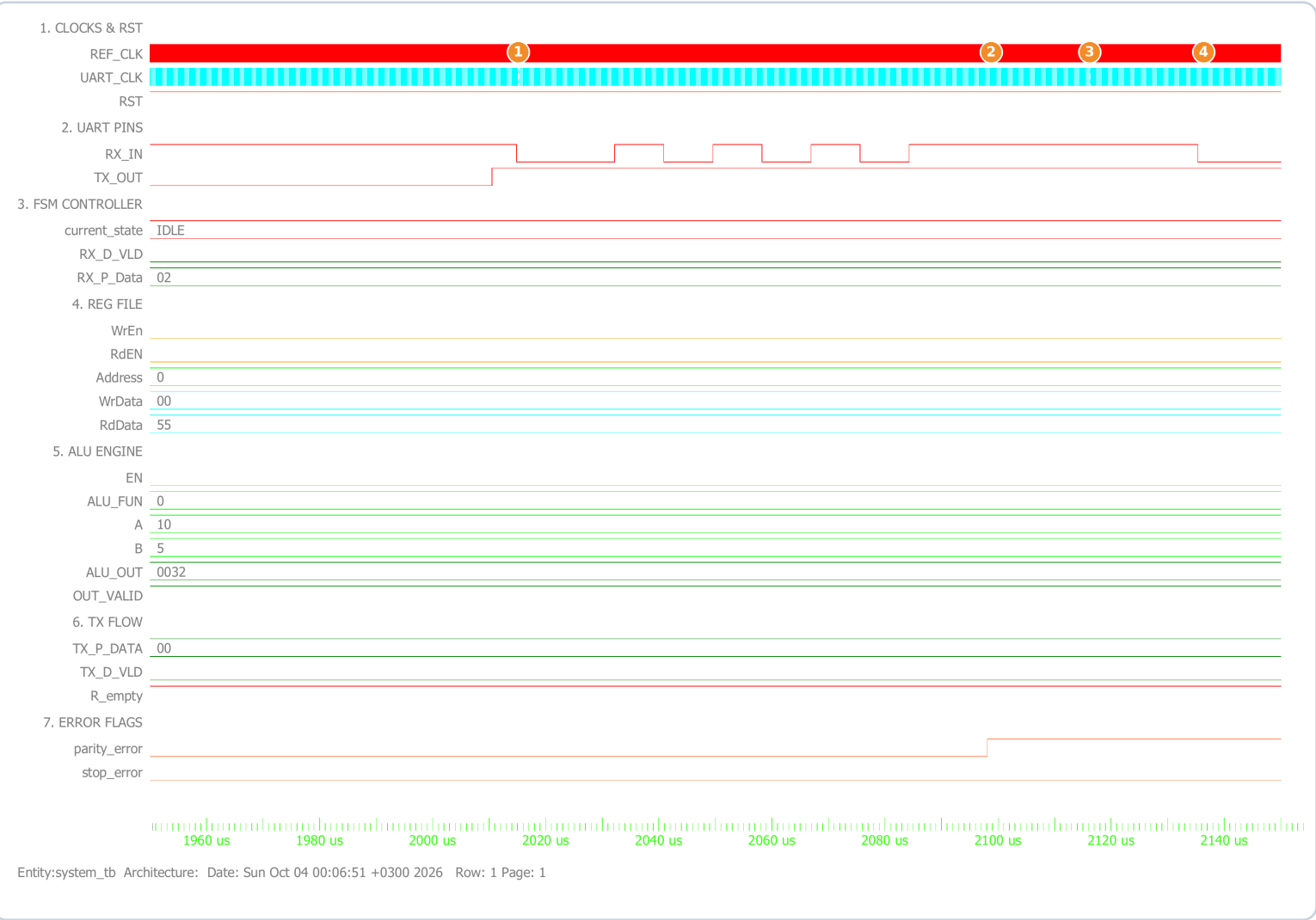
Time (us)	Event	Observed behaviour
~2015	Start bit of frame 0xAA	RX_IN falls; FSM_RX IDLE -> start -> str_t_check -> data; SYS_CTRL stays IDLE
~2023-2093	Data bits 0xAA (LSB first)	Deserializer collects 8 bits; RX_P_DATA is not forwarded (RX_P_Data stays 02 in the viewer)
~2098	Parity bit = 1 (expected 0)	FSM_RX parity -> parity_check; par_chk_en = 1: par_err = 1, parity_error goes high
~2102-2110	Stop bit = 1 (valid)	FSM_RX stop -> stop_check -> check; stp_err = 0
~2110	check state	par_err = 1: data_valid not generated; FSM_RX -> IDLE (frame dropped)
~2115	Testbench self-check	parity_error = 1 -> "[PASS] Parity Error Flag correctly asserted by DUT"
~2135	Next frame starts (Test 6)	RX_IN falls again (right edge of this window); parity_error stays high until the next parity check

RESULT 0xAA frame with wrong parity -> parity_error = 1, frame rejected, no command executed

Test 5 - Waveform

Annotated simulation output (system_tb)

Window ~1950 - 2150 us



Key events

1

~2015 us
Frame 0xAA starts on RX_IN (start bit) - parity bit will be corrupted

2

~2098 us
Parity bit sampled = 1 (expected 0) - parity_error goes high

3

~2115 us
Testbench check: parity_error = 1 -> PASS

4

~2135 us
Next frame (Test 6) starts on RX_IN

How to read this waveform

Sections 1-7 from top to bottom: clocks and reset, UART pins, SYS_CTRL FSM, register file, ALU engine, TX flow, error flags. Dashed orange lines mark the numbered events listed on the right. Here the frame on RX_IN is corrupted on purpose: watch parity_error / stop_error in section 7. current_state stays IDLE and RX_P_Data keeps its old value because the rejected frame never produces RX_D_VLD; TX_OUT stays idle high. The ALU and register values shown are leftovers from the previous tests.

Test 6 - Error injection: stop-bit error (0xAA, stop bit = 0)

Window ~2150 - 2306 us

Command definition, UART_RX FSM path, injected frame and event timeline

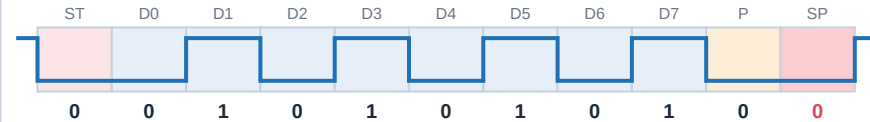
1. Command frame sent by the master (RX_IN)



0xAA = RF write command. It is never executed: the frame is rejected by the stop-bit check. Corrupted bit shown in red on the right.

3. Frame injected on RX_IN (corrupted bit highlighted)

Injected frame 0xAA: stop bit forced to 0 (expected 1)



Bits are shown LSB first. ST = start, P = parity, SP = stop. Idle line = 1.

2. UART_RX FSM_RX path (rejected frame)



SYS_CTRL: stays in IDLE - RX_D_VLD never pulses, the corrupted frame is dropped inside UART_RX (FSM_RX).

4. Verification checklist

- ✓ stop_error = 1 after the frame (~2227 us)
- ✓ parity_error returns to 0 (this frame has valid parity)
- ✓ No RX_D_VLD pulse - frame rejected
- ✓ SYS_CTRL stays IDLE: no Wren / RdEN / ALU EN
- ✓ No response frame on TX_OUT (R_empty stays high)
- ✓ Testbench self-check prints PASS

5. Event timeline (times read from the waveform, in us)

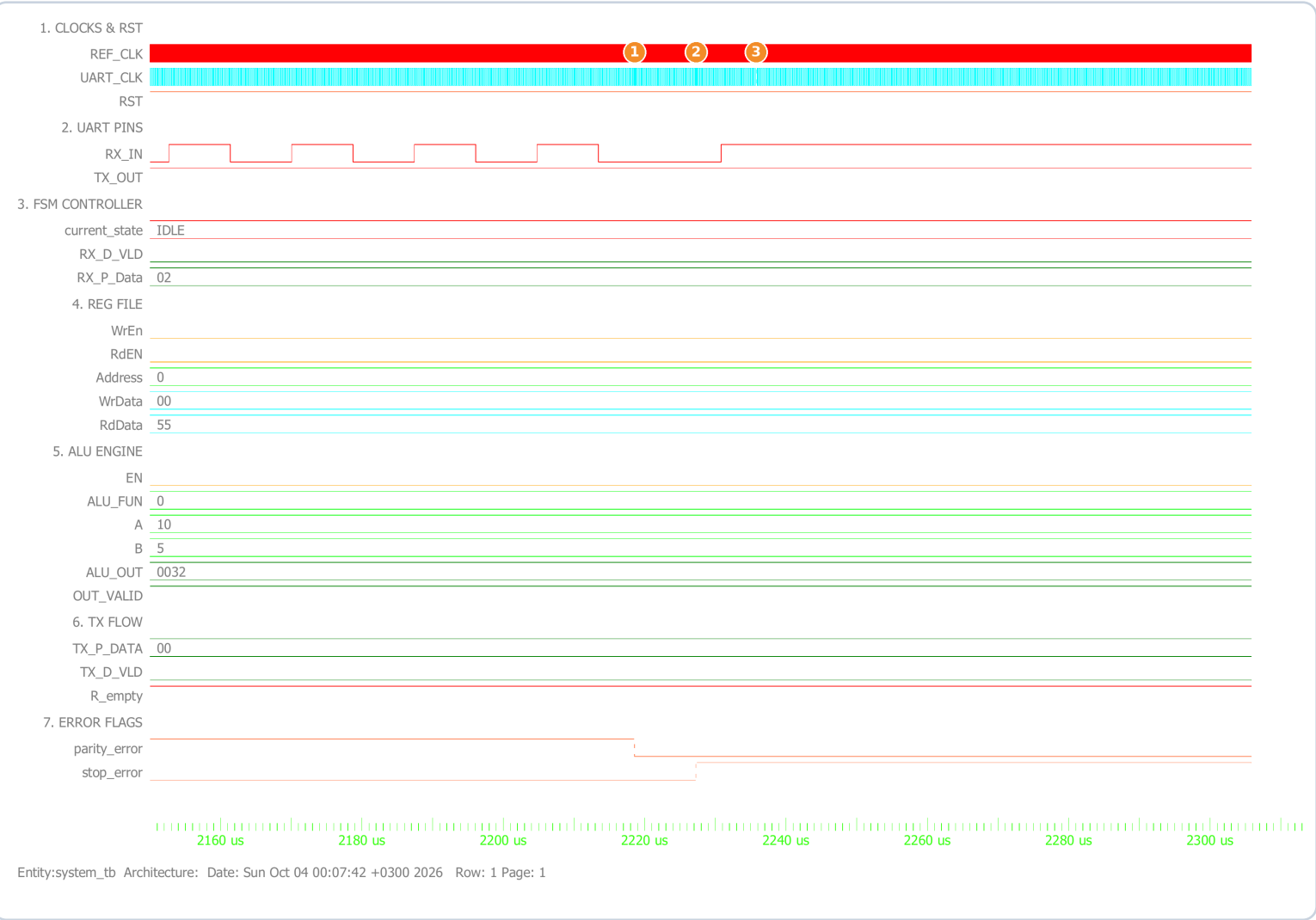
Time (us)	Event	Observed behaviour
~2135	Start bit of frame 0xAA	RX_IN falls (see right edge of Test 5 window); FSM_RX IDLE -> start -> str_t_check -> data
~2213	Parity bit = 0 (valid)	FSM_RX parity -> parity_check; par_chk_en = 1: par_err = 0
~2219	parity_error falls	par_err is re-evaluated by this frame, so the flag from Test 5 clears
~2222-2231	Stop bit = 0 (expected 1)	RX_IN stays low through the stop slot, then returns to idle high
~2227	stop_error rises	FSM_RX stop -> stop_check; stp_chk_en = 1: stp_err = 1, stop_error goes high
~2231	check state	stp_err = 1: data_valid not generated; FSM_RX -> IDLE (frame dropped)
~2236	Testbench self-check	stop_error = 1 -> "[PASS] Stop Bit Error Flag correctly asserted by DUT"

RESULT 0xAA frame with stop bit = 0 -> stop_error = 1, frame rejected, no command executed

Test 6 - Waveform

Annotated simulation output (system_tb)

Window ~2150 - 2306 us



Key events

1

~2219 us
Parity of the new frame is valid - parity_error returns low

2

~2227 us
Stop bit sampled = 0 (expected 1) - stop_error goes high

3

~2236 us
Testbench check: stop_error = 1 -> PASS

How to read this waveform

Sections 1-7 from top to bottom: clocks and reset, UART pins, SYS_CTRL FSM, register file, ALU engine, TX flow, error flags. Dashed orange lines mark the numbered events listed on the right. Here the frame on RX_IN is corrupted on purpose: watch parity_error / stop_error in section 7. current_state stays IDLE and RX_P_Data keeps its old value because the rejected frame never produces RX_D_VLD; TX_OUT stays idle high. The ALU and register values shown are leftovers from the previous tests.